

Bansilal Ramnath Agarwal Charitable Trust's
Vishwakarma Institute of Technology, Pune-37

(An autonomous Institute of Savitribai Phule Pune University)



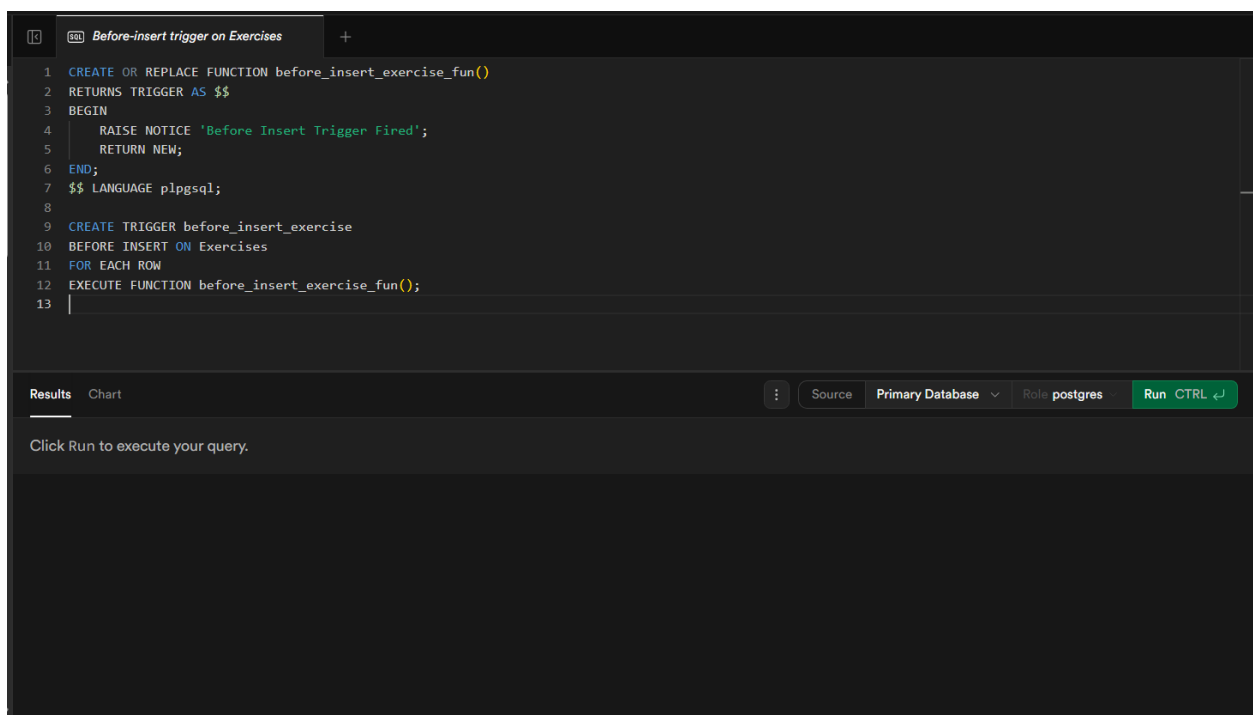
Department of Multidisciplinary
Engineering

Division	AIML-A
Batch	01
GR No.	12414243
Roll No.	08
Name	Abhishek Anil Patil

Title:

Write and execute all types of database triggers in PL/SQL.

Description: In this assignment, various types of database triggers were created and executed on the existing tables to understand how automatic actions work in response to data changes. We implemented BEFORE and AFTER triggers for all three operations: INSERT, UPDATE, and DELETE. Each trigger was linked to a function and executed automatically whenever the corresponding SQL operation was performed. This demonstrated how triggers can be used to monitor changes, validate data, log activities, and maintain consistency in the database without manual execution. The assignment helped in understanding the internal automation capabilities of SQL through real-time trigger execution.

Screenshots/Output:

```
1 CREATE OR REPLACE FUNCTION before_insert_exercise_fun()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     RAISE NOTICE 'Before Insert Trigger Fired';
5     RETURN NEW;
6 END;
7 $$ LANGUAGE plpgsql;
8
9 CREATE TRIGGER before_insert_exercise
10 BEFORE INSERT ON Exercises
11 FOR EACH ROW
12 EXECUTE FUNCTION before_insert_exercise_fun();
13 |
```

Results Chart

Source Primary Database Role postgres Run CTRL ↵

Click Run to execute your query.

14 After-Insert Trigger for Exercises +

```
1 CREATE OR REPLACE FUNCTION after_insert_exercise_fun()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     RAISE NOTICE 'After Insert Trigger Fired';
5     RETURN NEW;
6 END;
7 $$ LANGUAGE plpgsql;
8
9 CREATE TRIGGER after_insert_exercise
10 AFTER INSERT ON Exercises
11 FOR EACH ROW
12 EXECUTE FUNCTION after_insert_exercise_fun();
13
```

Results Chart

Source Primary Database Role postgres Run CTRL ↵

Click Run to execute your query.

14 Before-Update Trigger for Exercises +

```
1 CREATE OR REPLACE FUNCTION before_update_exercise_fun()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     RAISE NOTICE 'Before Update Trigger Fired';
5     RETURN NEW;
6 END;
7 $$ LANGUAGE plpgsql;
8
9 CREATE TRIGGER before_update_exercise
10 BEFORE UPDATE ON Exercises
11 FOR EACH ROW
12 EXECUTE FUNCTION before_update_exercise_fun();
13
```

Results Chart

Source Primary Database Role postgres Run CTRL ↵

Click Run to execute your query.

14

After-update trigger on Exercises

+

```
1 CREATE OR REPLACE FUNCTION after_update_exercise_fun()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     RAISE NOTICE 'After Update Trigger Fired';
5     RETURN NEW;
6 END;
7 $$ LANGUAGE plpgsql;
8
9 CREATE TRIGGER after_update_exercise
10 AFTER UPDATE ON Exercises
11 FOR EACH ROW
12 EXECUTE FUNCTION after_update_exercise_fun();
13
```

Results Chart

Source Primary Database Role postgres Run CTRL ↵

Click Run to execute your query.

14

Before-Delete Trigger on Exercises

+

```
1 CREATE OR REPLACE FUNCTION before_delete_exercise_fun()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     RAISE NOTICE 'Before Delete Trigger Fired';
5     RETURN OLD;
6 END;
7 $$ LANGUAGE plpgsql;
8
9 CREATE TRIGGER before_delete_exercise
10 BEFORE DELETE ON Exercises
11 FOR EACH ROW
12 EXECUTE FUNCTION before_delete_exercise_fun();
13
```

Results Chart

Source Primary Database Role postgres Run CTRL ↵

Click Run to execute your query.

After-Delete Trigger on Exercises

1

CREATE OR REPLACE FUNCTION after_delete_exercise_fun()

2

RETURNS TRIGGER AS \$\$

3

BEGIN

4

RAISE NOTICE 'After Delete Trigger Fired';

5

RETURN OLD;

6

END;

7

\$\$ LANGUAGE plpgsql;

8

9

CREATE TRIGGER after_delete_exercise

10

AFTER DELETE ON Exercises

11

FOR EACH ROW

12

EXECUTE FUNCTION after_delete_exercise_fun();

13

Results

Chart

Source

Primary Database

Role postgres

Run CTRL ↵

Click Run to execute your query.